

作业题 11 实验报告

姓 名: 王 继 勇
学 号: PB21511884
计算物理 A
(秋季, 2023)

中国科学技术大学
核科学技术学院

2023 年 11 月 6 日

目 录

1	题目	3
2	代码流程	3
2.1	第一部分 (DLA)	3
2.2	第二部分 (DBM)	6
2.2.1	随机行走 DBM	6
2.2.2	自规避随机行走 DBM	11
3	结果与分析	16

1 题目

模拟 2 维 DLA 以及介电击穿 (DBM) 图案并讨论

2 代码流程

2.1 第一部分 (DLA)

```
import numpy as np
import matplotlib.pyplot as plt
import multiprocessing
import time
from concurrent.futures import ThreadPoolExecutor
from numba import njit

aaaaaaaa = 16807
m = 2147483647

def azmodm(z):
    global m, aaaaaaaaa
    q = int(m / aaaaaaaaa)
    r = m % aaaaaaaaa
    tmp = aaaaaaaaa * (z % q) - r * int(z / q)
    if tmp >= 0:
        return tmp
    else:
        return tmp + m

bbbssbbb = 0
aaasssaaa = 0
ccssssccc = 0

def random01():
    global bbbssbbb, aaasssaaa, m, ccssssccc
    z1 = 668461
    z2 = 816815
    z3 = 116560
    z4 = 4813144
    if aaasssaaa == 0 and ccssssccc == 0:
        bbbssbbb = z1
    if aaasssaaa == m - 10000000 and ccssssccc == 0:
        bbbssbbb = z2
        aaasssaaa = 1
        ccssssccc = 1
    if aaasssaaa == m - 10000000 and ccssssccc == 1:
        bbbssbbb = z3
        aaasssaaa = 1
```

```

        cccsssccc = 2
    if aaasssaaa == m - 10000000 and cccsssccc == 2:
        bbbsssbbb = z4
        aaasssaaa = 1
        cccsssccc = 3
    bbbsssbbb = azmodm(bbbsssbbb)
    aaasssaaa += 1
    return bbbsssbbb / m

L = 10000 ##L*L网格
# xi = 3 ##速度参数
# fai0 = 100 ##边界电势
xy = np.zeros((L, L, 2), dtype='float64') ##第三维为 (是否占据 0/1, 历史路劲 0/1)
xy[int(L/2), int(L/2)] = (1, 0)
maxL=3
N1=[]
N2=[]
def calcu(indices):
    global maxL ,N1,N2
    # print(1)
    ii,jj= indices
    if (xy[ii][jj][0]==1) :
        tmp=(ii-int(L/2))**2+(jj-int(L/2))**2
        if tmp>=maxL**2:
            maxL=int(np.sqrt(tmp))+2
            N1.append(maxL) ##更新maxL-n图
            N2.append(n)

def maxxy():
    with ThreadPoolExecutor() as executor:
        indices = [(i, j) for i in range(int(L/2)-(maxL+5),int(L/2)+(maxL+5)) for j in range(int(L/2)-(maxL+5),int(L/2)+(maxL+5))]
        executor.map(calcu, indices)

def addone():
    global xy
    maxxy() ##更新最大距离
    xy[:, :, 1]=0 ###初始化历史路劲
    iftouch=0
    fai=random01()
    I=round((maxL+5)*np.cos(2*np.pi*fai)+L/2)
    J=round((maxL+5)*np.sin(2*np.pi*fai)+L/2)
    xy[I][J][1]=1
    while not(iftouch):
        # print(maxL)
        a = random01()
        # lastx=I
        # lasty=J

```

```

        # print(I,J,ii,jj)
        if a <= 0.25:
            I += 1
        elif 0.25 < a <= 0.5:
            I -= 1
            # print(a)
        elif 0.5 < a <= 0.75:
            J += 1
        elif 0.75 < a <= 1:
            J -= 1
            # print(a)
        # print(I,J)
        if ((I-int(L/2))**2+(J-int(L/2))**2>=(maxL*5)**2) or xy[I][J][0]==1 or
(I <= 1) or (I == L-2) or (J < 1) or (J == L-2):    ## (xy[I-1][J][1]+xy[I+1][J][1]+xy[I][J-1][1]+xy[I][J+1][1])
        # print(I,J)
        fai=random01()
        # print(2)
        I=round((maxL+5)*np.cos(2*np.pi*fai)+L/2)
        J=round((maxL+5)*np.sin(2*np.pi*fai)+L/2)
        # xy[:, :, 1]=0 ###初始化历史路劲
        # xy[I][J][1]=1
        continue
    # if xy[I][J][1]==1:
    #     I=lastx
    #     J=lasty
    #     print(3)
    #     continue
    # else:
    #     xy[I][J][1]=1
    #     print(4)
    iftouch=sum(sum(xy[I-1:I+2,J-1:J+2,0])) ##判断有无接触
    if (iftouch):
        xy[I][J][0]=1
        # print(I,J)

def change(arr):
    x = []
    y = []
    for i in range(len(arr)):
        x.append(arr[i][0])
        y.append(arr[i][1])
    x = np.array(x)
    y = np.array(y)
    return x, y

def findarr():
    arr=np.array([[int(L/2),int(L/2)]])
    for i in range(int((L/2)-(maxL+10)),int((L/2)+(maxL+5))):
        for j in range(int((L/2)-(maxL+10)),int((L/2)+(maxL+5))):
            if xy[i][j][0]==1:

```

```

        arr=np.concatenate((arr,np.array([[i,j]])),axis=0)  ##将已占据的点的坐标加入数组
    return arr

n=0
nn=0
plt.ion()  # 开启交互模式
plt.subplots()
for k in range(3000):
    addone()
    n=sum(sum(xy[:,0]==1))
    print(n)
    plt.clf()      # 清空画布
    if(nn!=maxL):
        nn=maxL
        plt.scatter(N2,N1)
        plt.pause(3)
        plt.clf()      # 清空画布
    plt.xlim(int((L/2)-(maxL+20)),int((L/2)+(maxL+20)))
    plt.ylim(int((L/2)-(maxL+20)),int((L/2)+(maxL+20)))
    arr=findarr()
    x, y = change(arr)
    plt.scatter(x, y,s=1)
    plt.pause(0.2)
    # plt.clf()      # 清空画布

plt.ioff()
plt.show()

# plt.figure()
# plt.scatter(N2,N1)
# plt.show()

```

2.2 第二部分 (DBM)

2.2.1 随机行走 DBM

```

import numpy as np
import matplotlib.pyplot as plt
import multiprocessing
import time
from concurrent.futures import ThreadPoolExecutor
from numba import njit

aaaaaaaaa = 16807
m = 2147483647

def azmodm(z):

```

```

    global m, aaaaaaaaa
    q = int(m / aaaaaaaaa)
    r = m % aaaaaaaaa
    tmp = aaaaaaaaa * (z % q) - r * int(z / q)
    if tmp >= 0:
        return tmp
    else:
        return tmp + m

bbbssbbb = 0
aaasssaaa = 0
ccssssccc = 0

def random01():
    global bbbssbbb, aaasssaaa, m, ccssssccc
    z1 = 666955
    z2 = 816815
    z3 = 116560
    z4 = 4813144
    if aaasssaaa == 0 and ccssssccc == 0:
        bbbssbbb = z1
    if aaasssaaa == m - 10000000 and ccssssccc == 0:
        bbbssbbb = z2
        aaasssaaa = 1
        ccssssccc = 1
    if aaasssaaa == m - 10000000 and ccssssccc == 1:
        bbbssbbb = z3
        aaasssaaa = 1
        ccssssccc = 2
    if aaasssaaa == m - 10000000 and ccssssccc == 2:
        bbbssbbb = z4
        aaasssaaa = 1
        ccssssccc = 3
    bbbssbbb = azmodm(bbbssbbb)
    aaasssaaa += 1
    return bbbssbbb / m

N = 100 ##系综数
L = 30 ##L*L网格
xi = 3 ##速度参数
fai0 = 100 ##边界电势
xy = np.zeros((L, L, 3), dtype='float64') ##第三维为 (是否占据 0/1, 电势, 历史路劲 0/1)
xy[int(L/2)-1:int(L/2)+1, int(L/2)-1:int(L/2)+1] = (1, fai0, 0)

# @njit
def walk( indices):
    ii, jj= indices
    if xy[ii][jj][0]==0:
        # xy[ii][jj][2]=1
        # print(2)
        tmp = np.zeros(N)

```

```

for k in range(N):
    I, J = ii, jj
    # xy[:, :, 2]=0 ###初始化历史路劲
    while xy[I][J][0] != 1:
        a = random01()
        # lastx=I
        # lasty=J
        # print(I,J,ii,jj)
        if a <= 0.25:
            I += 1
        elif 0.25 < a <= 0.5:
            I -= 1
        elif 0.5 < a <= 0.75:
            J += 1
        else:
            J -= 1
        # print(I,J)
    if (I < 0) or (I == L) or (J < 0) or (J == L):
        I = -1
        # print("触碰边界")
        break

    # if xy[I][J][2]==1:
    #     I=lastx
    #     J=lasty
    #     # print("回到上一部")
    # elif (1<=I<=L-2) & (1<=J<=L-2) & (xy[I][J][2]!=1) :###未到边界
    #     # if xy[I-1][J][2]+xy[I+1][J][2]+xy[I][J-1][2]+xy[I][J+1][2]==4: ##死胡同
    #         # I,J=ii,jj ###回到原点
    #         # xy[:, :, 2]=0 ###初始化历史路劲
    #         # print("回到原点")
    #     # else:
    #         # xy[I][J][2]=1
    #         # print("成功走一步")
    #     # elif not((1<=I<=L-2) & (1<=J<=L-2)) & (xy[I][J][2]!=1): ##到了边界
    #         # xy[I][J][2]=1
    #         # print("到了边界")
    if not(I == -1): ##不是触碰边界
        tmp[k] = fai0
    xy[ii][jj][1] = np.mean(tmp)

def updatexy():
    global xy, N, fai0
    with ThreadPoolExecutor() as executor:
        indices = [(i, j) for i in range(L) for j in range(L)]
        executor.map(walk, indices)

def calcul(indicas):

```



```

x,y=indicas
global xy
global p
if xy[x][y][0]==0:
    tmp=0
    if x==0:
        tmp+=xy[x+1][y][0] #####判断有无到达边界，到达以后只计算另一边是否接触
        if y==0:
            tmp+=xy[x][y+1][0]+xy[x+1][y+1][0]
        elif y==L-1:
            tmp+=xy[x][y-1][0]+xy[x+1][y-1][0]
        else:
            tmp+=xy[x][y+1][0]+xy[x][y+1][0]+xy[x+1][y+1][0]+xy[x+1][y+1][0]
    elif x==L-1:
        tmp+=xy[x-1][y][0]
        if y==0:
            tmp+=xy[x][y+1][0]+xy[x-1][y+1][0]
        elif y==L-1:
            tmp+=xy[x][y-1][0]+xy[x-1][y-1][0]
        else:
            tmp+=xy[x][y+1][0]+xy[x][y+1][0]+xy[x-1][y+1][0]+xy[x-1][y+1][0]
    else:
        tmp+=xy[x-1][y][0]+xy[x+1][y][0]
        if y==0:
            tmp+=xy[x][y+1][0]+xy[x+1][y+1][0]+xy[x-1][y+1][0]
        elif y==L-1:
            tmp+=xy[x][y-1][0]+xy[x+1][y-1][0]+xy[x-1][y-1][0]
        else:
            tmp+=xy[x][y+1][0]+xy[x][y-1][0]+xy[x+1][y+1][0]+xy[x+1][y-1][0]+xy[x-1][y+1][0]+xy[x-1][y-1][0]
    if tmp>0:
        p[x][y][0]=1
        p[x][y][1]=tmp ###周围的点数
        p[x][y][2]=tmp*(fai0-xy[x][y][1])**xi

p =np.zeros((L, L, 3), dtype='float64')
n=0
def updatep():
    global p,n
    p[:,:]=(0,0,0)
    n=sum(sum(xy[:,:,0]==1))
    with ThreadPoolExecutor() as executor:
        indices = [(i, j) for i in range(L) for j in range(L)]
        executor.map(calcu, indices)
    sumv=sum(sum(p[:,:,2]))
    p[:,:,2]=p[:,:,2]/sumv

def change(arr):
    x = []
    y = []
    for i in range(len(arr)):

```

```

        x.append(arr[i][0])
        y.append(arr[i][1])
    x = np.array(x)
    y = np.array(y)
    return x, y

def findarr():
    arr=np.array([[int(L/2),int(L/2)]])
    for i in range(L):
        for j in range(L):
            if xy[i][j][0]==1:
                arr=np.concatenate((arr,np.array([[i,j]])),axis=0)
    return arr

plt.ion() # 开启交互模式
plt.subplots()
# plt.xlim(0, L)
# plt.ylim(0, L)
# x, y = change(findarr())
# ax = plt.scatter(x, y)
for k in range(500):
    updatexy()
    updatep()
    miu=random01()
    tmp=0
    for i in range(L):
        for j in range(L):
            tmp+=p[i][j][2]
            if miu<=tmp:
                xy[i][j]=(1,fai0,0)
                break
        else:
            continue
    else:
        continue
    break
    print(n+1)
    plt.clf() # 清空画布
    plt.xlim(0, L)
    plt.ylim(0, L)
    arr=findarr()
    x, y = change(arr)
    plt.scatter(x, y,s=3)
    plt.pause(0.2)
plt.ioff()
plt.show()

```

2.2.2 自规避随机行走 DBM

最初模拟时，由于运行速度太慢，尝试在计算电势时使用自规避随机行走，理由是，随机行走时粒子回到历史路劲上某处后相当于最后这一段没有走，不如直接绕过，但是实际结果反而比随机行走慢（注意自规避的可执行文件里系综数只有 50 了，但每走一步的时间仍然比系综数为 100 的随机行走长），经过考虑，原因可能有以下：

1. 自规避行走时粒子碰到历史路劲后会返回上一步，这样大大增加了模拟次数；
2. 自规避行走时粒子碰到死胡同会回到原点，也会导致大量模拟次数；
3. 存在历史路劲以后，粒子想要到达边界的路劲被限制，必须拐许多弯路

```
import numpy as np
import matplotlib.pyplot as plt
import multiprocessing
import time
from concurrent.futures import ThreadPoolExecutor
from numba import njit

aaaaaaaa = 16807
m = 2147483647

def azmodm(z):
    global m, aaaaaaaaa
    q = int(m / aaaaaaaaa)
    r = m % aaaaaaaaa
    tmp = aaaaaaaaa * (z % q) - r * int(z / q)
    if tmp >= 0:
        return tmp
    else:
        return tmp + m

bbbssbbb = 0
aaasssaaa = 0
ccssssccc = 0

def random01():
    global bbbssbbb, aaasssaaa, m, ccssssccc
    z1 = 666955
    z2 = 816815
    z3 = 116560
    z4 = 4813144
    if aaasssaaa == 0 and ccssssccc == 0:
        bbbssbbb = z1
    if aaasssaaa == m - 10000000 and ccssssccc == 0:
        bbbssbbb = z2
        aaasssaaa = 1
```

```

        cccssssccc = 1
    if aaasssaaa == m - 10000000 and cccssssccc == 1:
        bbbssssbbb = z3
        aaasssaaa = 1
        cccssssccc = 2
    if aaasssaaa == m - 10000000 and cccssssccc == 2:
        bbbssssbbb = z4
        aaasssaaa = 1
        cccssssccc = 3
    bbbssssbbb = azmodm(bbbssssbbb)
    aaasssaaa += 1
    return bbbssssbbb / m

N = 50 ##系数
L = 30 ##L*L网格
xi = 3 ##速度参数
fai0 = 100 ##边界电势
xy = np.zeros((L, L, 3), dtype='float64') ##第三维为 (是否占据 0/1, 电势, 历史路劲 0/1)
xy[int(L/2)-1:int(L/2)+2, int(L/2)-1:int(L/2)+2] = (1, fai0, 0)

# @njit
def walk( indices):
    ii,jj= indices
    if xy[ii][jj][0]==0:
        xy[ii][jj][2]=1
        # print(2)
        tmp = np.zeros(N)
        for k in range(N):
            I, J = ii, jj
            xy[:, :, 2]=0 ###初始化历史路劲
            while xy[I][J][0] != 1:
                a = random01()
                lastx=I
                lasty=J
                # print(I,J,ii,jj)
                if a <= 0.25:
                    I += 1
                elif 0.25 < a <= 0.5:
                    I -= 1
                elif 0.5 < a <= 0.75:
                    J += 1
                else:
                    J -= 1
                # print(I,J)
            if (I < 0) or (I == L) or (J < 0) or (J == L):
                I = -1
                # print("触碰边界")
                break
            if xy[I][J][2]==1:
                I=lastx
                J=lasty

```

```

        # print("回到上一部")
    elif (1<=I<=L-2) & (1<=J<=L-2) & (xy[I][J][2]!=1) :###未到边界
        if xy[I-1][J][2]+xy[I+1][J][2]+xy[I][J-1][2]+xy[I][J+1][2]==4: ##死胡同
            I,J=ii,jj    ###回到原点
            xy[:, :, 2]=0 ###初始化历史路劲
            # print("回到原点")
        else:
            xy[I][J][2]=1
            # print("成功走一步")
        elif not((1<=I<=L-2) & (1<=J<=L-2)) & (xy[I][J][2]!=1): ##到了边界
            xy[I][J][2]=1
            # print("到了边界")
        if not(I == -1): ##不是触碰边界
            tmp[k] = fai0
    xy[ii][jj][1] = np.mean(tmp)

def updatexy():
    global xy, N, fai0
    with ThreadPoolExecutor() as executor:
        indices = [(i, j) for i in range(L) for j in range(L)]
        executor.map(walk, indices)

def calcu(indicas):
    x,y=indicas
    global xy
    global p
    if xy[x][y][0]==0:
        tmp=0
        if x==0:
            tmp+=xy[x+1][y][0]
            if y==0:
                tmp+=xy[x][y+1][0]+xy[x+1][y+1][0]
            elif y==L-1:
                tmp+=xy[x][y-1][0]+xy[x+1][y-1][0]
            else:
                tmp+=xy[x][y+1][0]+xy[x][y-1][0]+xy[x+1][y+1][0]+xy[x+1][y-1][0]
        elif x==L-1:
            tmp+=xy[x-1][y][0]
            if y==0:
                tmp+=xy[x][y+1][0]+xy[x-1][y+1][0]
            elif y==L-1:
                tmp+=xy[x][y-1][0]+xy[x-1][y-1][0]
            else:
                tmp+=xy[x][y+1][0]+xy[x][y-1][0]+xy[x-1][y+1][0]+xy[x-1][y-1][0]
        else:
            tmp+=xy[x-1][y][0]+xy[x+1][y][0]
            if y==0:
                tmp+=xy[x][y+1][0]+xy[x+1][y+1][0]+xy[x-1][y+1][0]

```

```

        elif y==L-1:
            tmp+=xy[x][y-1][0]+xy[x+1][y-1][0]+xy[x-1][y-1][0]
        else:
            tmp+=xy[x][y+1][0]+xy[x][y-1][0]+xy[x+1][y+1][0]+xy[x+1][y-1][0]+xy[x-1][y+1][0]+xy[x-1][y-1][0]
    if tmp>0:
        p[x][y][0]=1
        p[x][y][1]=tmp
        p[x][y][2]=tmp*(fai0-xy[x][y][1])**xi

p =np.zeros((L, L, 3), dtype='float64')
n=0
def updatep():
    global p,n
    p[:,:]=0,0,0
    n=sum(sum(xy[:,:,0]==1))
    with ThreadPoolExecutor() as executor:
        indices = [(i, j) for i in range(L) for j in range(L)]
        executor.map(calcu, indices)
    sumv=sum(sum(p[:,:,2]))
    p[:,:,2]=p[:,:,2]/sumv

def change(arr):
    x = []
    y = []
    for i in range(len(arr)):
        x.append(arr[i][0])
        y.append(arr[i][1])
    x = np.array(x)
    y = np.array(y)
    return x, y

def findarr():
    arr=np.array([[int(L/2),int(L/2)]])
    for i in range(L):
        for j in range(L):
            if xy[i][j][0]==1:
                arr=np.concatenate((arr,np.array([[i,j]])),axis=0)
    return arr

plt.ion() # 开启交互模式
plt.subplots()
# plt.xlim(0, L)
# plt.ylim(0, L)
# x, y = change(findarr())
# ax = plt.scatter(x, y)
for k in range(500):
    updatexy()

```

```

updatep()
miu=random01()
tmp=0
for i in range(L):
    for j in range(L):
        tmp+=p[i][j][2]
        if miu<=tmp:
            xy[i][j]=(1,fai0,0)
            break
        else:
            continue
    else:
        continue
    break
print(n)
plt.clf()      # 清空画布
plt.xlim(0, L)
plt.ylim(0, L)
arr=findarr()
x, y = change(arr)
plt.scatter(x, y,s=3)
plt.pause(0.2)
plt.ioff()
plt.show()

```

可以看到，使用：

```

def updatexy():
    global xy, N, fai0
    with ThreadPoolExecutor() as executor:
        indices = [(i, j) for i in range(L) for j in range(L)]
        executor.map(walk, indices)

```

来进行并行运算，时间大大缩短了。

3 结果与分析

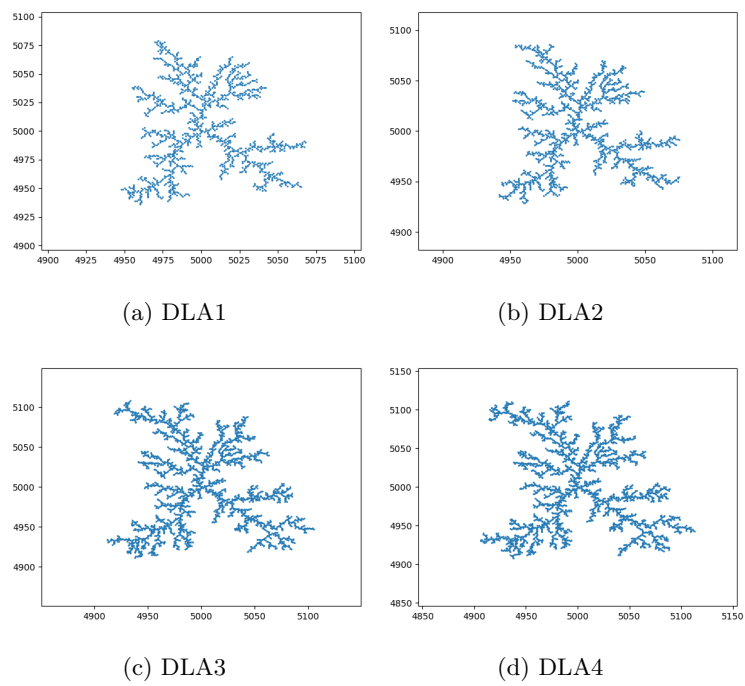


图 1: DLA 生长结果图

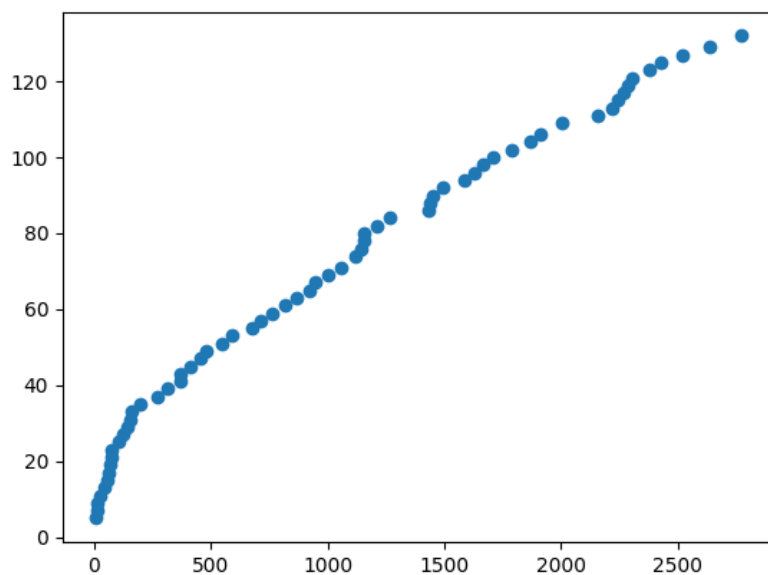
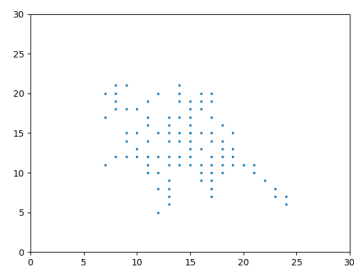
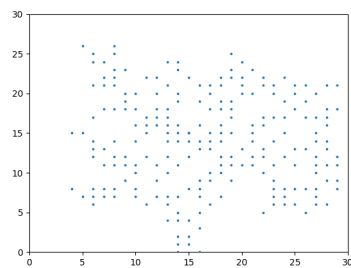


图 2: 最长半径 $\max L$ (y 轴) 与点数 n (x 轴) 的关系

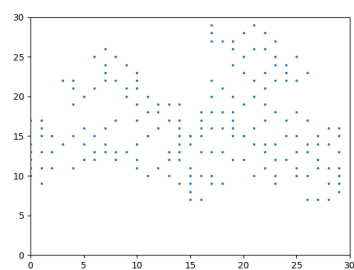
由图 2 可以看出, n 较小时, $\max L$ 随着 n 增加快速增加, n 较大时, $\max L$ 与 n 近似为线性关系, 但斜率在慢慢下降。理论上斜率是应该下降的, 因为 $\max L$ 越大, 分叉越多, 空间越大, 粒子越不容易到最尖端。



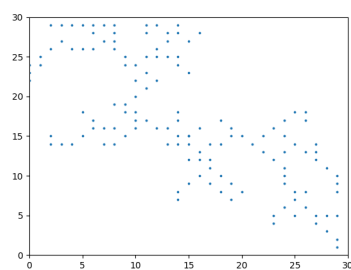
(a) $\xi=0.1$



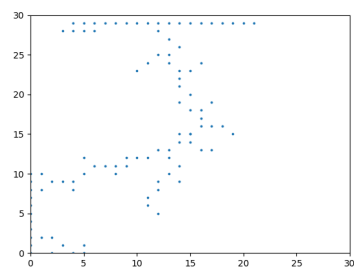
(b) $\xi=0.5$



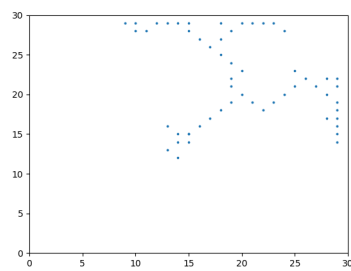
(c) $\xi=1$



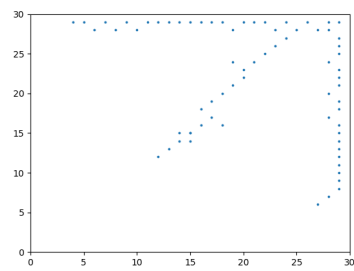
(d) $\xi=2$



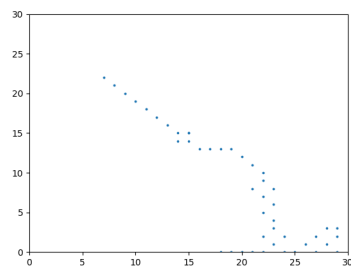
(e) $\xi=3$



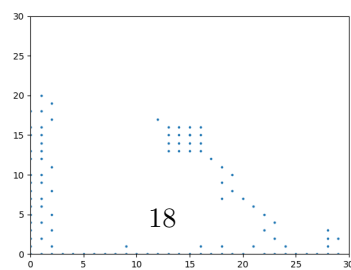
(f) $\xi=5$



(g) $\xi=8$



(h) $\xi=10$, 初始 1



(i) $\xi=10$, 初始 2

图 3: DBM 结果图

由结果可知，速率参数 (ξ) 越大，图形分叉越少，击穿速度越大；而初始位置被占据格点越多，击穿速度也越快。